

TravelLink Backend — COMPLETE

Last Updated: 2026-05-31 | Status: Backend 100% Done — Frontend Next

Stack: .NET 8, EF Core, SQL Server, SignalR, Gemini AI, Cloudinary, Open-Meteo

Project Structure

```
Server/
├── Controllers/    AuthController, UserController, FriendController,
│                  GroupController, ExpenseController,
ItineraryController
├── Services/       Interface + Implementation for each module
├── Models/Entities/ 12 entity models
├── DTOs/           Organized by module (Auth, Expenses, Friends,
Groups, Itinerary, Location, Users)
├── Data/           AppDbContext
├── Hubs/           LocationHub (SignalR)
├── Common/         ServiceResult<T>, ItineraryCategories
└── wwwroot/        test-location.html (SignalR test page)
```

Module 1 — Authentication

Service: AuthService | Controller: AuthController

- JWT-based login & registration
 - Password hashing, token generation with `ClaimTypes.NameIdentifier` (Guid userId)
-

Module 2 — User

Controller: UserController

- Get user profile, Update profile (name, bio, etc.)
- Profile image upload via Cloudinary (`ImageService`)

✓ Module 3 — Friend Management

Service: FriendService | **Controller:** FriendController

Endpoint	Description
Send friend request	Creates FriendRequest entity
Accept / Reject	Updates request status
Unfriend	Blocked if unsettled 1-on-1 expenses exist (group expenses exempt)
List friends	Returns accepted connections

Key constraints:

- UnfriendAsync checks `s.Expense.GroupId == null` — only personal unsettled splits block unfriending
- AreFriendsAsync helper used across modules

✓ Module 4 — Group Management

Service: GroupService | **Controller:** GroupController

Endpoint	Description
Create group	Creator auto-assigned as Admin
Update group info	Admin only
Upload group cover image	Cloudinary
Add/Remove members	Admin only
Leave group	Auto-deletes group if last member leaves
List group members	

✓ Module 5 — Image Storage

Service: `ImageService` | Uses **Cloudinary**

- User profile pictures, Group cover pictures
- Upload returns URL stored in DB
- Old image deleted before new upload

✓ Module 6 — Expense Module

Service: `ExpenseService` | **Controller:** `ExpenseController`

Endpoint	Description
Add expense	Equal or Custom split, friends-only for 1-on-1
Delete expense	Only payer can delete; blocked if others have settled
Settle split	<code>MarkSplitAsPaidAsync</code> — idempotent (double-tap safe)
Get expense by ID	Auth check — payer, participant, or group member
Get group expenses	All expenses for a group
Get group balance	Net debt simplification algorithm
Analytics dashboard	Spend by category, per-person, settled vs unsettled
User-to-user summary	Transaction history + net balance between two users

Key design:

- `ServiceResult<T>` wrapper — structured error messages for frontend toasts
- Friends-only constraint enforced at service level (not frontend)
- `MarkSplitAsPaidAsync` returns `Ok(true)` if already paid — idempotent ✓

✓ Module 7 — Real-Time Location

Hub: `LocationHub` (SignalR) | Route: `/hubs/location`

Hub Method	Description
------------	-------------

<code>JoinGroupSession(groupId)</code>	Auth → auto-leave old session → join group room
<code>JoinPrivateSession(targetUserId)</code>	Friend check → auto-leave old → join private room
<code>SendLocation(dto)</code>	Broadcast coords to all others in session
<code>LeaveSession()</code>	Voluntarily exit, notify others
<code>OnDisconnectedAsync</code>	Auto-cleanup on crash/close

Design decisions:

- `static ConcurrentDictionary` — shared state across all hub instances
- Pure relay — no DB writes per location update (stateless, scalable)
- One active session per user enforced server-side
- JWT via query string for WebSocket (`OnMessageReceived` in `Program.cs`)
- Test page: `wwwroot/test-location.html` with Leaflet map



Module 8 — Smart Itinerary Builder

Service: `ItineraryService` (~790 lines) | **Controller:** `ItineraryController`

External APIs: Gemini 1.5 Flash (AI), Open-Meteo (weather + geocoding)

Entities

Entity	Key Fields
<code>ItineraryRequest</code>	GroupId, Destination, StartDate, EndDate, TotalBudget, VehicleType, VehicleCount, IsRental, DailyRentalCostPerVehicle, DailyFuelCostPerVehicle, DailyHotelCostPerRoom, NumberOfRooms, GroupSize, Status
<code>PlaceSuggestion</code>	ItineraryRequestId, Name, Type, Notes, AdminApproved
<code>SuggestionVote</code>	SuggestionId, UserId
<code>GeneratedItinerary</code>	ItineraryRequestId, TotalDays, DroppedSuggestionsJson
<code>ItineraryDay</code>	ItineraryId, DayNumber, Date, Title, WeatherNote

ItineraryItem	DayId, PlaceName, Category, Description, DurationMinutes, EstimatedCostPerPerson, BookingSearchQuery, BookingType, IsCompleted
---------------	--

Request Status Lifecycle

"Open" → "Generating" → "Generated"

Endpoints (11 total)

Endpoint	Method	Description
POST /api/itinerary/request	Admin	Create request (opens suggestion phase). Blocked if any request exists.
DELETE /api/itinerary/request/{requestId}	Admin	Full cascade delete
POST /api/itinerary/{requestId}/suggest	Members	Add place suggestion
DELETE /api/itinerary/suggestion/{id}	Owner/Admin	Delete suggestion
POST /api/itinerary/suggestion/{id}/vote	Members	Toggle vote
PATCH /api/itinerary/suggestion/{id}/review	Admin	Approve/reject
GET /api/itinerary/{requestId}/suggestions	Members	List suggestions with vote counts
POST /api/itinerary/{requestId}/generate	Admin	Generate or re-generate (auto-deletes old if already generated)
GET /api/itinerary/{requestId}/result	Members	Full itinerary result

PATCH <code>/api/itinerary/item/{itemId}/complete</code>	Members	Toggle item complete
POST <code>/api/itinerary/request/auto-generate</code>	Admin	Skip suggestions — generate instantly. Replaces any existing request automatically.
GET <code>/api/itinerary/group/{groupId}/status</code>	Members	Get current itinerary status for a group (returns null if none)

Two Flows, One Engine

Collaborative: POST `/request` → suggest → vote → review → POST `/generate`
 Auto: POST `/request/auto-generate` (returns result directly)

Both call the shared `private async Task RunGenerationAsync(request, note)`.

RunGenerationAsync Steps

1. Weather fetch (Open-Meteo geocoding → forecast)
2. Suggestions query (empty for auto-generate → AI picks freely)
3. Budget math:
 - `totalDays = EndDate - StartDate + 1`
 - `numberOfNights = totalDays - 1`
 - `tripVehicleCost = (dailyRental + dailyFuel) × vehicleCount × totalDays`
 - `tripHotelCost = DailyHotelCostPerRoom × NumberOfRooms × numberOfNights`
 - `remainingBudget = TotalBudget - tripVehicleCost - tripHotelCost`
4. Prompt construction (vehicle context + accommodation context + suggestions + weather + budget)
5. Gemini AI call → JSON response
6. Strip markdown fences, parse JSON
7. Persist: `GeneratedItinerary` → `ItineraryDay[]` → `ItineraryItem[]`

8. `Status = "Generated"` , `ChangeTracker.Clear()` (prevents EF cache issues in auto-flow)

AI Prompt Context Rules

Scenario	AI Instruction
Own/rental vehicle	"Vehicle cost pre-budgeted — do NOT add transport items"
Public transport	"Include bus/auto/taxi as separate items with realistic cost"
Hotel provided	"Hotel pre-budgeted (₹X/night × N rooms) — do NOT add hotel items"
Free accommodation	"Free accommodation — do NOT add hotel items"

ItineraryResultDto Fields

```
Id, RequestId, Destination, StartDate, EndDate, TotalDays, GroupSize,
GeneratedAt
TotalBudget, VehicleTotalCost, DailyVehicleCost, VehicleType,
VehicleCount
HotelTotalCost, DailyHotelCost, NumberOfNights
EstimatedActivityCost, EstimatedTotalSpend, BudgetRemaining,
EstimatedCostPerPerson
ReplacedExisting (bool – true if old itinerary was auto-deleted and
replaced)
Days[] → Items[] (booking info, cost, category, completion status)
DroppedSuggestions[]
```

GroupItineraryStatusDto Fields

```
RequestId, Destination, StartDate, EndDate, TotalBudget
Status (Open | Generating | Generated)
TotalSuggestions, ApprovedSuggestions
```

Frontend uses `Status` to decide which screen to show. Returns `null` Data if no request exists yet.

Vehicle Auto-Calculation

If `VehicleCount` not provided, auto-computed from `VehicleType` + group size:

Vehicle	Capacity
Scooty / Bike	2 per vehicle
Car	4 per vehicle
SUV	7 per vehicle
Traveller	1 vehicle total

Key Design Patterns

- `DeleteRequestCoreAsync(Guid)` — private cascade delete helper (votes → suggestions → items → days → itinerary → request). Used by both `DeleteItineraryRequestAsync` (public, admin-checked) and `AutoGenerateItineraryAsync` internally.
- `ReplacedExisting` flag stamped onto result DTO **after** `GetItineraryResultAsync` returns — frontend shows replacement warning toast.
- `ChangeTracker.Clear()` after auto-generate to prevent EF identity map returning stale in-memory Days collection.
- Date validation guard in both `CreateRequestAsync` and `AutoGenerateItineraryAsync`: `EndDate.Date <= StartDate.Date → Fail`.

Common Infrastructure

`ServiceResult<T>` (`Common/ServiceResult.cs`)


```
public class ServiceResult<T>
{
    public bool Success { get; set; }
    public string? Message { get; set; }
    public T? Data { get; set; }
    public static ServiceResult<T> Ok(T data) => new() { Success = true,
Data = data };
    public static ServiceResult<T> Fail(string message) => new() {
Success = false, Message = message };
}
```

Used by all services. Controllers pattern: `return result.Success ? Ok(result) : BadRequest(result)`.

ItineraryCategories (Common/ItineraryCategories.cs)

Static class with category constants: `Sightseeing, Food, Transport, Accommodation, Activity, Shopping, Nature, Culture, Rest, NightLife`. `PromptList` property injected into Gemini prompt.

All Migrations Applied

- Initial schema
- `FixItineraryDayAndItemSchema`
- `UpdateItinerarySchemaV2` (vehicle fields)
- `AddHotelFieldsToItineraryRequest`

Next: Angular Frontend

What to Build

Screen	Key Features
Auth	Login / Register with JWT storage
Friends	Search users, send/accept/reject requests, unfriend

Groups	Create, manage members, cover image upload
Expenses	Add expense, view splits, settle, balance dashboard
Location	Leaflet map with live SignalR markers (up to 5 users)
Itinerary	Collaborative flow (suggest → vote → generate) + Auto-generate flow

Frontend Notes

- Angular 17+ (standalone components)
- SignalR client: `@microsoft/signalr`
- Map: Leaflet + `leaflet` npm package
- JWT stored in `localStorage`, sent as `Authorization: Bearer <token>` header
- SignalR token passed via query string `?access_token=<token>`
- Backend CORS configured for `http://localhost:4200`
- `ReplacedExisting` flag in itinerary response → show replacement toast
- `GroupItineraryStatusDto.Status` → route user to correct itinerary screen

Bugs Fixed (All Sessions Combined)

Bug	Fix
<code>totalDays</code> date subtraction reversed	<code>EndDate - StartDate + 1</code>
<code>dailyVehicleCost = rental × fuel</code>	Changed to <code>rental + fuel</code>
<code>dailyHotelCost</code> ignored <code>NumberOfRooms</code>	<code>DailyHotelCostPerRoom × NumberOfRooms</code>
Missing <code>await</code> on final <code>SaveChangesAsync</code>	Added <code>await</code>
EF identity map: auto-generate returned <code>Days.Count = 0</code>	<code>ChangeTracker.Clear()</code>
<code>dto.Note / requestId</code> scope in <code>RunGenerationAsync</code>	Changed to <code>note</code> param / <code>request.Id</code>

Status = "Generating" missing in AutoGenerateItineraryAsync	Added to request initializer
Missing DB migration for hotel columns	Add-Migration + Update-Database
MarkSplitAsPaidAsync — double-tap returned error	Changed to Ok(true) if already paid
GenerateItineraryAsync — blocked re-generation	Now auto-clears old itinerary and re-runs
AutoGenerateItineraryAsync — allowed duplicate requests	Now deletes existing request first
Stray class-level properties in ItineraryService	Removed
Missing semicolon on date guard in CreateRequestAsync	Fixed
RegenerateItineraryAsync — dead code	Removed (absorbed by GenerateItineraryAsync)